

Partie IV: JavaScript

JS - Introduction



- JavaScript (ECMA-262) 3^e langage web irremplaçable
- Langage de programmation du web, le plus populaire, différent de Java
- Sert à donner un comportement (dynamique) aux pages web
- Interaction, changement de: balises, attributs et style CSS
- Utiliser des `<script>` pour insérer du code JS en interne (HTML)
- Utiliser `<script src="path_file.js">` pour mettre le code JS dans fichier externe

JS - Pourquoi ? Comment ?

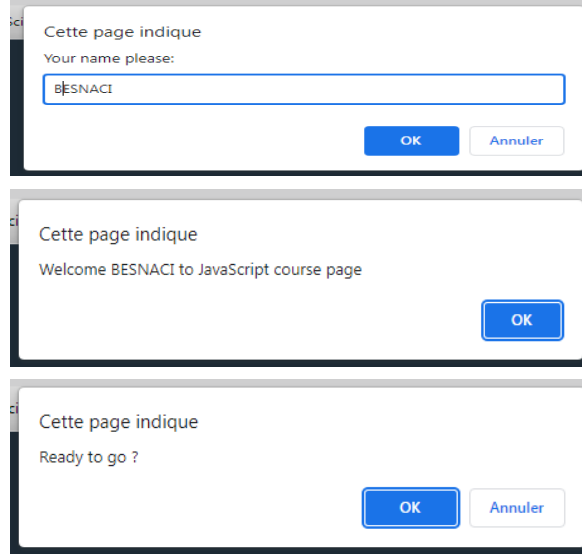
Internal/External

```
//HTML file
<!DOCTYPE html>
<html>
<body>
<h1>JavaScript</h1>
<p id="demo"></p>
<script>
var price1 = 5;
const price2 = 6;
let total = price1 + price2;
document.getElementById("demo").innerHTML =
"The total is: " + total;
</script>
</body>
</html>
```

```
//JS file
var price1 = 5;
const price2 = 6;
let total = price1 + price2;
document.getElementById("demo").innerHTML =
"The total is: " + total;
```

```
<script src="myScript.js"></script> //HTML file
```

Interaction



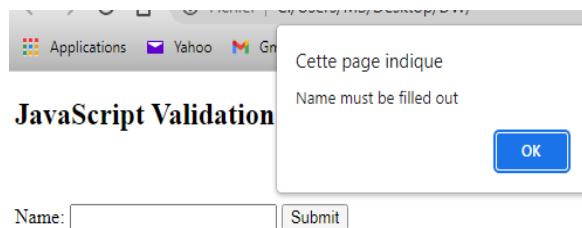
Control

JavaScript Validation

Please input a number between 1 and 10:

11 Submit

Input not valid



Modification

JavaScript

can change HTML content.

Click Me!



JavaScript

Hello JavaScript!

Click Me!

Contenu

JavaScript

can change HTML attribute (src of img tag) value.



light on

light off



JavaScript

can change HTML attribute (src of img tag) value.



light on

light off

Attribut

JavaScript

can change HTML elts style.

Click Me!



JavaScript

can change HTML elts style.

Click Me!

Style

JavaScript

can hide HTML elements.

MyList



JavaScript

can hide HTML elements.

MyList
first item
second item
last item

Cacher/
Afficher

JS – Variables & types

- Déclaration de variables: **var**, **let** et **const**
 - Après déclaration la valeur = *undefined*
 - Déclaration explicite pas obligatoire
 - **const**: définir une valeur fixe, initialization obligatoire
 - **let**: déclarer avant utiliser, block scope
 - **var**: déclarer même après utiliser, global scope
- Type de variable: number, string, boolean, array, object, function, ...
 - **implicite** et **dynamique**
 - Utiliser l'opérateur *typeof* pour connaître le type d'une valeur
 - **NaN** signifiant une quantité non numérique
 - **isNaN()** renvoie true si NaN et false sinon

```
let x = 5/"three";
```

```
<script>
let b = 3, c;
a = b;
const PI = 3.14;
var a;
d = 2*PI;
{let b = 0; d = 0;}
let e;
alert("b="+b+", d="+d+ " et e="+e);
</script>
```

www.w3schools.com indique
b=3, d=0 et e=undefined

OK

```
let x = 5;
y = x + 1;
x = "cinq";
alert("x="+x+", y="+y);
```

www.w3schools.com indique
x=cinq, y=6

OK

```
<!DOCTYPE html>
<html>
<body>
<h2>typeof Operator</h2>
<script>
x = typeof "john" + "<br>" +
  typeof 3.14 + "<br>" +
  typeof false + "<br>" +
  typeof [1,2,3,4] + "<br>" +
  typeof function () {};
document.write(x);
</script>
</body>
</html>
```

typeof Operator

string
number
boolean
object
function

JS – instructions de base

- **Expressions**: combinaison d'opérateurs (+, -, &&, !, ==, ===, ==!, >=, ?, **typeof**, ...) et d'opérandes (littéraux, variables, fonctions) – syntaxe classique
- **Affectation**: =, +=, *=, % =, ...
- **E/S**: boîtes de dialogue, sorties sur le document HTML, sur la console du navigateur
 - Boîtes de dialogue: **alert()**, **confirm()**, **prompt()**
 - Sur HTML: **document.write()**, l'attribut **innerHTML** (fonctions de DOM)
 - Sur la console: **console.log()**
- **Commentaires** // une seule ligne, /* multilignes */
- JS est sensible à la casse (identifiants)

```
3 <body>
4 <p id="i1"></p>
5 <script>
6 /* Exemples
7 d'instructions
8 de base */
9 let x = prompt("Enter x:");
10 let y = prompt("Enter y:");
11 x /= (y+3)%2;
12 alert("x="+x);
13 console.log("x est de type" + typeof x);
14 y = x;
15 x += "";
16 alert("x est de type:"+typeof x);
17 document.getElementById("i1").innerHTML =
18 "Le type de x n'a pas changé:" + x===y;
19 //document.write("...") est utilisé pour tester
20 </script>
21 </body>
```

JS – instructions de contrôle

- Conditionnel

- **if** (Cond) Inst / **if** (Cond) Inst **else** Inst → classique
- **switch** (Exp) {**case** Val: Inst break; ... **default**: Inst} → comparaison avec ===

- Répétition

- **for** (Init; Cond; Incr) Inst → classique
- **for** (Var **in** Obj) Inst → itérer sur les propriétés d'un objet
- **for** (Var **of** Itér) Inst → itérer sur les valeurs d'un itérable
- **while** (Cond) Inst → classique
- **do** Inst **while** (Cond) → classique

```
1 <script>
2   if (hour < 18) { alert("Good day");}
3   else { alert("Good evening");}
4
5   switch (new Date().getDay()) {
6     case 0: alert("Sunday"); break;
7     case 1: alert("Monday");break;
8     case 2: alert("Tuesday");break;
9     default: alert("a day");}
10
11   let text = "";
12   const cars = ["BMW", "Volvo", "Saab", "Ford", "Fiat", "Audi"];
13   for (let i = 0; i < cars.length; i++) {
14     text += cars[i] + "<br>";
15   }
16   const person = {fname:"John", lname:"Doe", age:25};
17   text = "";
18   for (let x in person) {
19     text += person[x];}
20
21   const cars = ["BMW", "Volvo", "Mini"];
22
23   text = "";
24   for (let x of cars) {
25     text += x;}
26 </script>
```

JS – fonctions, évènements, erreurs

- **Fonctions**

- **function** Name(P1, ...) {code}
- Fonction avec **return** pour renvoyer des valeurs
- Appel **sans parenthèses** pour renvoyer le **code**
- **Typeof** renvoie **function** pour les fonctions

- **Évènements**

- Contrôler des évènements produits par le **navigateur** ou **l'utilisateur**
 - Ex: Chargement/fermeture de la page, ...
 - Ex: Remplissage de données, déclenchement d'actions, ...
- HTML propose des attributs pour les gérer
 - Ex: onload, onchange, onclick, onmouseover, onkeypress, ...

- **Erreurs**

- **try** {code} **catch**(err) {code}
- Contrôler un code pouvant causer une erreur (try)
- Supporter l'erreur en exécutant un certain code (catch)

```
1  <p id="usr">User name here !</p>
2  <input type="button" onclick="insertName()">
3  <script>
4  function insertName(){
5  try {
6      const n = prompt("Name:");
7      document.getElementById("user").innerHTML = n;
8  } catch (error) {
9      alert(error.message);
10 }
11 }
12 </script>
```

JS – objets & classes

- **Objets**

- **const** obj = {prop1:val1, ...propi: **function**(...){...}, ... }
 - Ex: const car = {type:"Fiat", model:"500", color:"white"};
- obj.prop = val / obj["prop"] = val / obj.propi(...)
 - Ex: alert(car.type); car["model"] = "100"; car.move();

- **Classes**

- **class** cname { constructor(param) { Init } m1(...) { ... } ... }
- **constructor**: création d'objets et initialisation de propriétés
- **const** obj = **new** cname(...); création d'un objet de la classe
- Propriétés **implicites** dans le constructeur
- obj **instanceof** type: true si obj est instance de type

```
1 <script>
2 const person = {
3   firstName: "dev",
4   lastName : "Web",
5   id       : 5566,
6   fullName : function() {
7     return this.firstName + " " + this.lastName;
8   }
9 };
10 person.id++;
11 person["firstName"] = "Dev";
12 person.age = 20;
13
14 class Car {
15   constructor(name, year) {
16     this.name = name;
17     this.year = year;
18   }
19   age() {
20     let date = new Date();
21     return date.getFullYear() - this.year;
22   }
23 }
24 let myCar = new Car("Ford", 2014);
25 alert("My car is " + myCar.age() + " years old.");
26 </script>
```

JS – Chaines (String object)

- **Chaînes**: utilisées pour stocker et manipuler du texte (" " / ' ')
- **Caractères spéciaux**: \", \', \\, \n, \t, ...
- **Expressions régulières** (regex) : chaîne représentant un modèle de recherche
 - **Syntaxe**: `/pattern/modifiers` → ex: `/algeria(-|_)\d*/i` ex: `/[0-5]+\s-\s(da|dt)/g`
- **Longueur**: `str.length`
- **Méthodes**:
 - `charAt(idx)`: caractère d'indice "idx"
 - `indexOf(str)`: indice du 1er occurrence d'une chaîne "str"
 - `match(rex)`: tableau d'occurrences d'une chaîne/regex "rex" (ou null)
 - `replace(rex,str)`: remplace les occurrences d'une chaîne/regex "rex" par une autre chaîne "str"
 - `search(rex)`: indice du 1er occurrence (ou -1) d'une chaîne/regex "rex"
 - `slice(idx,lng)`: partie de la chaîne commençant de l'indice "idx" de longueur "lng"
 - `split(sep)`: tableau de sous-chaînes suivant le séparateur "sep"
 - `trim()`: chaîne sans caractères espace, tab, ... aux extrémités
 - ...

JS – Chaines (String object)

```
1  <!DOCTYPE html>
2  <html>
3  <body>
4  <script>
5      var text = " Ceci, est un exemple de \'texte\'. \n";
6      alert(text[1]+text.charAt(2)); //Ce
7      alert(text.indexOf("ex")); //14
8      alert(text.slice(7,10)); //est
9      alert(text.trim()); //Ceci, est un exemple de 'texte'.
10     let tm = text.match(/[a-z]*e(x|s)[a-z]*/ig);
11     alert(tm); //est,exemple,texte
12     let rt = text.replace(/[a-z]+ /g,"-");
13     alert(rt); //Ceci, ----'texte'.
14     let st = text.search(/\sex/);
15     alert("Indice du 1er ' ex': "+ st); //13
16 </script>
17 </body>
18 </html>
```

JS – Tableaux (Array object)

- **Création**: `const t=[1, "deux", 3.0];` / `const t=[]; t[0]=1;t[1]="deux";t[2]=3.0;` / `const t=new Array(1,"deux",3.0);`
- **Taille** (nbr d'élts): `t.length`
- **Ajout d'élts**: `t[t.length]=val;` / `t.push(val);`
- **Vérification**: `Array.isArray(t)`
- **Méthodes**:
 - `t.push(elt)/t.pop(), t.unshift(elt)/t.shift()`: empiler/dépiler au début/fin
 - `t.join(sep)`: liste des elts séparés par sep
 - `t1.concat(t2, ...)`: concatination de plusieurs tableaux
 - `t.slice(i,n)`: partie allant de l'indice i avec n-1 elt
 - `t.splice(i,n,e1,...)`: remplacer n elt à partir de l'indice i par les elts ei (renvoie les elts remplacés et t est changé)
 - `t.sort()/t.reverse()`: tri alphanumérique ascendant/descendant
 - `t.indexOf(e)`: indice de la 1ere occurrence de l'elt e
 - `t.forEach(f)`: répéter l'appel d'une fonction f pour chaque elt
 -

JS – Tableaux (Array object)

```
4  <script>
5      const a = [1, "deux", 3.0];
6      const b = []; b[0]=1; b[1]="deux"; b[2]=3.0;
7      const c = new Array(1,"deux",3.0);
8      alert(a); alert(b); alert(c); //1,deux,3
9      let size = a.length;
10     a[size] = 4.1; a.push(5);
11     alert(a); //1,deux,3,4.1,5
12     a.unshift("zero"); a.pop();
13     alert(a.join(" ")); //zero 1 deux 3 4.1
14     let bc = b.concat(c);
15     alert("slice: "+bc.slice(1,4)); //deux,3,1
16     bc.splice(1,4,"x","y","z");
17     alert("splice: "+bc); //splice: 1,x,y,z,3
18     alert(bc.sort()); //1,3,x,y,z
19     let d = bc.sort(); d.push(3);
20     alert(bc.indexOf(3)); //1
21     var elts=""; a.forEach(parenth);
22     function parenth(item){elts += item+" / ";}
23     alert(elts); //zero / 1 / deux / 3 / 4.1 /
24 </script>
```

JS – Date (Date object)

- **Création**: `const d=new Date();` / `const d=new Date(y,m,d,h,mt,s,ms);` / `const d=new Date(str);` / `const d=new Date(ms_1970)`
- **Formats (chaines)**:
 - **ISO**: YYYY-MM-DD (**2022-03-31**), YYYY-MM (**2022-03**), YYYY (**2022**), YYYY-MM-DDTHH:MM:SSZ (**2022-03-31T21:05:23Z**)
 - **Short**: MM/DD/YYYY (**15/06/2002**)
 - **Long**: MMM DD YYYY (**Apr 15 2000**), MMM... DD YYYY (**April 10 2015**)
- **Affichage**: utiliser `d.toString()` pour un affichage plus agréable
- **Méthodes**:
 - `Date.parse(str)`: ms d'une date valide **ex: `Date.parse("March 21, 2012")` → 1332284400000**
 - `d.getFullYear()`, `d.getMonth()`, `d.getDate()`, `d.getHours()`, ..., `d.getTime()`, `d.getDay()`, `Date.now()`: donner respectivement l'année, le mois, le jours, l'heure, ..., le temps en ms, le jours de la semaine et le temps actuel
 - `d.setFullYear()`, ...: affecter respectivement l'année, ...

JS – Math (Math object)

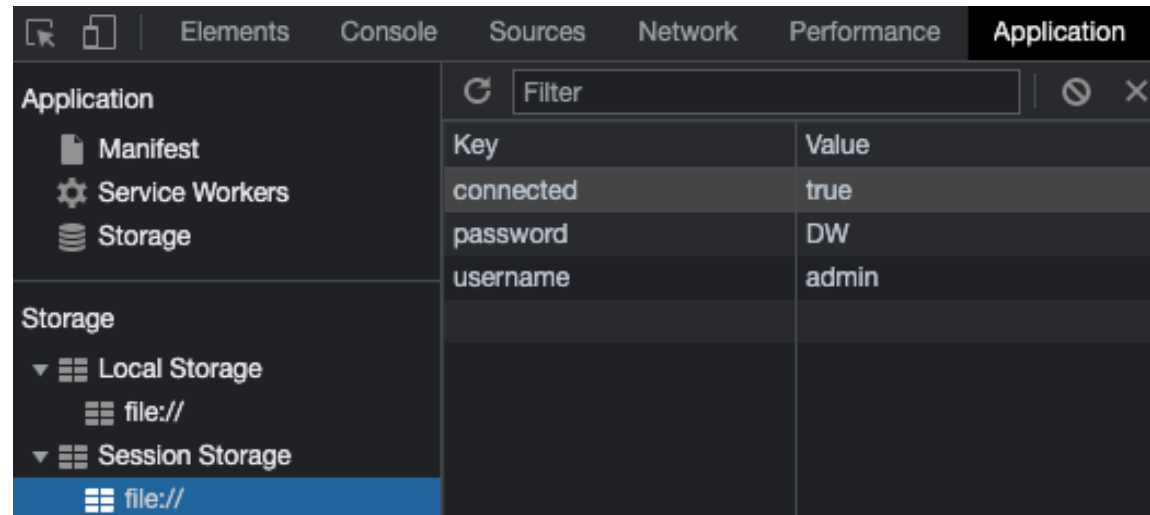
- **Math** est un objet statique, **Math.PI**, **Math.E**, **Math.LN2**, ... sont des constantes
- **Méthodes**
 - **Math.round(x)**: x arrondi à son entier le plus proche
 - **Math.trunc(x)**: partie entière de x
 - **Math.sign(x)**: si x est positif (1), négatif (-1) ou null (0)
 - **Math.power(x,y)**: puissance y de x
 - **Math.sqrt(x)**: racine carrée de x
 - **Math.abs(x)**: valeur absolue de x
 - **Math.sin/cos(x)**: sinus/cosinus de x
 - **Math.min/max(x1,x2, ...)**: min/max d'une liste d'arguments
 - **Math.random()**: nombre aléatoire entre 0 et 1
 - **Math.log(x)**: logarithme népérien de x
 - ...

JS – Data Storage objects

- Créer et manipuler des données en local sur le navigateur pour un site web particulier
- Forme de données: couples « key, value »
- Durée: permanente (**localStorage** object) ou provisoire (**sessionStorage** object)
 - `setItem("key","value")` → insertion
 - `getItem("key")` → récupération
 - `removeItem("key")` → suppression
 - `clear()` → suppression du tout

```
function login(){
    let un = document.getElementById("un").value;
    let pw = document.getElementById("pw").value;
    if ((un == "admin") && (pw == "DW")){
        sessionStorage.setItem("connected", "true");
        sessionStorage.setItem("username", un);
        sessionStorage.setItem("password", pw);
    }
}

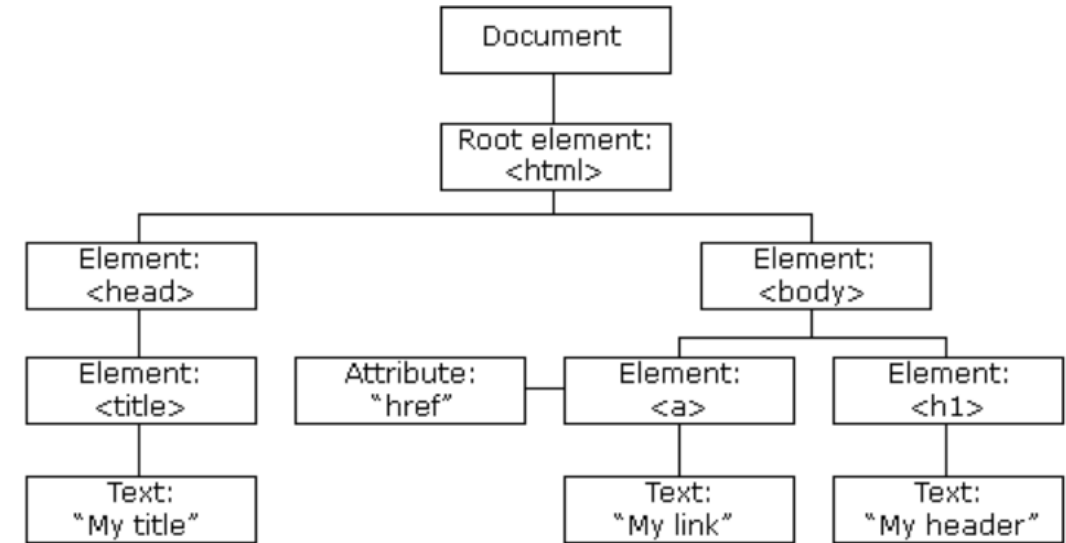
function logout(){
    sessionStorage.removeItem("connected");
}
```



Key	Value
connected	true
password	DW
username	admin

JS – DOM HTML

- DOM HTML: modèle d'objet **standard**, il définit :
 - Les éléments HTML en tant qu'objets
 - Les propriétés de tous les éléments HTML
 - Les méthodes pour accéder à tous les éléments HTML
 - Les événements pour tous les éléments HTML
- DOM HTML est une **norme** sur la manière d'obtenir, de modifier, d'ajouter ou de supprimer des éléments HTML.



```
1  <!DOCTYPE html>
2  <html>
3    <head>
4      <title>My title</title>
5    </head>
6    <body>
7      <h1>teMy header</h1>
8      <a href="address">My link</a>
9    </body>
10 </html>
```

JS – DOM HTML

- **Manipuler les élts HTML** revient à manipuler les objets DOM à travers leurs **propriétés** (ex: innerHTML) et **méthodes** (ex: getElementById)
- **Chercher des éléments**
 - document.getElementById(id)
 - document.getElementsByTagName(tag)
 - document.getElementsByClassName(class)
 - document.querySelectorAll(selector)
- **Modifier des éléments**
 - e.innerHTML = val
 - e.attribute_name = val
 - e.style.property_name = val
 - e.setAttribute(attribute_name, val)

```
1  <!DOCTYPE html>
2  <html>
3  <body>
4  <h2>JavaScript HTML DOM</h2>
5  <div>
6  |   <p>Text1</p>
7  |   <p class="intro" id="2nd">Text2</p>
8  </div>
9  <p class="intro">Text3</p>
10 <input type="checkbox" checked>
11 <script>
12 |   const a = document.getElementsByClassName("intro");
13 |   const b = document.getElementById("2nd");
14 |   const c = document.getElementsByTagName("p")
15 |   const d = document.querySelectorAll("div>p")
16 |   a[1].innerHTML = c[0].innerHTML + " + " + b.innerHTML;
17 |   b.style.background = "lightblue";
18 |   document.getElementsByTagName("input")[0].checked = false;
19 </script>
20 </body>
21 </html>
```

JS – DOM HTML

- Ajout/suppression d'éléments
 - document.createElement(e)
 - document.removeChild(e)
 - document.appendChild(e)
 - document.replaceChild(new,old)
- Ajouter gestionnaire d'évènement
 - e.onclick = function(){code}
 - e.addEventListener(event, function)

```
1  <!DOCTYPE html>
2  <html>
3  <body>
4  <h2>JavaScript HTML DOM</h2>
5  <div>
6  |   <p>Text1</p>
7  </div>
8  <p>Text2</p>
9  <input type="button" value="ok">
10 <script>
11     const divn = document.getElementsByTagName("div")[0];
12     const pn = document.createElement("h4");
13     pn.innerHTML = "Example";
14     divn.appendChild(pn);
15     const im = document.createElement("img");
16     im.setAttribute("src", "logo.png");
17     divn.replaceChild(im,divn.firstElementChild);
18     const lastp = document.body.children[2];
19     document.body.removeChild(lastp);
20     const btn = document.getElementsByTagName("input")[0];
21     btn.onclick = function(){btn.value = "clicked ...";}
22     btn.addEventListener("mouseover", change());
23     function change(mess){btn.value = "Mouse overed ...";}
24 </script>
25 </body>
26 </html>
```

JS – DOM HTML

- **Éléments HTML spécifiques**

- document.documentElement, document.body, document.head, document.title
- document.forms, document.images
- document.URL, document.domain

- **Naviguer entre les éléments**

- e.parentNode, e.childNodes[n°], e.firstChild, e.lastChild, e.nextSibling, e.previousSibling
- e.nodeName, e.nodeValue, e.nodeType

```
1  <!DOCTYPE html>
2  <html>
3      <head>
4          <title>Old title</title>
5      </head>
6  <body>
7      <h2>JavaScript HTML DOM</h2>
8      <form action="" name="myForm">
9          Name: <input type="text" name="nom"> <br>
10         Email: <input type="text" name="courriel">
11     </form>
12     <script>
13         document.title = "New title";
14         document.body.style.background = "lightblue";
15         document.forms.myForm.nom.placeholder = "Name";
16         document.forms.myForm.courriel.placeholder = "email";
17         alert("Domain: "+document.domain+"URL: "+document.URL);
18         const mf = document.forms.myForm;
19         document.forms.myForm.innerHTML +=
20             "<br> first label is: "+
21             mf.childNodes[0].nodeValue+", next tag is: "+
22             mf.childNodes[0].nextSibling.nodeName+
23             "<br> parent tag is:"+
24             mf.children[0].parentNode.nodeName;
25     </script>
26 </body>
27 </html>
```